

--snip--

```
'92': 92, '93': 93, '94': 94, '96': 96, '97': 97, '98': 98, '99': 99}
```

A set comprehension ❶ uses braces instead of square brackets and produces a set value. A dictionary comprehension ❷ produces a dictionary value and uses a colon to separate the key and value in the comprehension.

These comprehensions are concise and can make your code more readable. But notice that the comprehensions produce a list, set, or dictionary based on an iterable object (in this example, the `range` object returned by the `range(100)` call). Lists, sets, and dictionaries are iterable objects, which means you could have comprehensions nested inside of comprehensions, as in the following example:

```
>>> nestedIntList = [[0, 1, 2, 3], [4], [5, 6], [7, 8, 9]]
>>> nestedStrList = [[str(i) for i in sublist] for sublist in nestedIntList]
>>> nestedStrList
[['0', '1', '2', '3'], ['4'], ['5', '6'], ['7', '8', '9']]
```

But nested list comprehensions (or nested set and dictionary comprehensions) cram a lot of complexity into a small amount of code, making your code hard to read. It's better to expand the list comprehension into one or more `for` loops instead:

```
>>> nestedIntList = [[0, 1, 2, 3], [4], [5, 6], [7, 8, 9]]
>>> nestedStrList = []
>>> for sublist in nestedIntList:
...     nestedStrList.append([str(i) for i in sublist])
...
>>> nestedStrList
[['0', '1', '2', '3'], ['4'], ['5', '6'], ['7', '8', '9']]
```

Comprehensions can also contain multiple `for` expressions, although this tends to produce unreadable code as well. For example, the following list comprehension produces a flattened list from a nested list:

```
>>> nestedList = [[0, 1, 2, 3], [4], [5, 6], [7, 8, 9]]
>>> flatList = [num for sublist in nestedList for num in sublist]
>>> flatList
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```
